

Mapping the Duality of Memory in Sequence Models and the Brain

Prashant Rangarajan

Paul G. Allen School of Computer Science and Engineering

University of Washington, Seattle, WA

`prashr@cs.washington.edu`

General Examination Report

Doctor of Philosophy

University of Washington

2026

Reading Committee:

Rajesh P. N. Rao, Chair

Andrea Stocco

Sewoong Oh

Aniruddh Vashisth

University of Washington

Abstract

Mapping the Duality of Memory
in Sequence Models and the Brain

Prashant Rangarajan

Paul G. Allen School of Computer Science and Engineering

University of Washington, Seattle, WA

`prashr@cs.washington.edu`

Chair of the Supervisory Committee:

Prof. Rajesh P. N. Rao

Computer Science and Engineering

Memory serves two complementary functions in both artificial and biological systems: reducing context size by compressing past tokens into a bounded internal “working” memory, and enabling fast generalization through binding and episodic memory. Recent computational neuroscience has characterized the duality of these two systems in the brain: a synaptic weight-based system in the hippocampus for episodic memory, and an activity-based slot system in prefrontal cortex for working memory. This report reviews both literatures in three parts. The first covers memory architectures in sequence models, including state-space models, hybrid quadratic-linear systems, retrieval-augmented systems, and external slot memories in goal-directed agents. The second covers biological memory systems, including cognitive maps in the hippocampal-entorhinal system. The final part discusses key-value memory as a unifying framework, and proposes directions for open questions, such as how real-time experience is selectively transformed into episodic and working memory, the trade-off between explicit context and internal memory, how memories consolidate into structural knowledge and schemas, and how these principles bear on continual learning and long-horizon world modeling.

TABLE OF CONTENTS

	Page
Chapter 1: Introduction	1
Chapter 2: Memory Architectures in Sequence Models	3
2.1 Outer-product memory: one primitive, many names	5
2.2 State-space duality: Mamba-2	6
2.3 Blending complementary memory systems	10
2.4 Memory as test-time learning	12
2.5 Retrieval-augmented generation: external symbolic memory	12
2.6 Memory in goal-directed agents: MERLIN	14
Chapter 3: Biological Memory Systems	18
3.1 Cognitive maps as learned positional codes: TEM and TEM-t	19
3.2 The duality of synaptic and activity-based memory	21
3.3 Parallelizing the cognitive map: MapFormer	23
Chapter 4: Bridging Biological and Machine Memory	26
4.1 Shared mathematical primitives	26
4.2 Key-value memory in the brain	28
Chapter 5: Future Directions	32
Chapter 6: Conclusion	38

Chapter 1

INTRODUCTION

It is tempting to treat memory as storage, keeping every past item in an external store and retrieving it on demand, and large retrieval-augmented systems do exactly this and work well for many tasks. But the memory systems that matter most, in both machines and brains, do more than store and retrieve. They serve two purposes at once: compressing the past into a bounded internal state, so that a model or an animal carries forward only what the current computation needs, and binding new experience into structured, addressable traces that support rapid generalization to situations never seen before. The brain appears to split this labour by substrate, holding episodic memory in hippocampal synaptic weights and working memory in sustained prefrontal activity, with key-value memory proposed to unify the two. This report maps how much of the machinery behind these two functions is shared between brains and machines.

The survey takes both fields in turn before drawing them together around the primitives they share. On the machine-learning side it examines state-space models [Dao and Gu, 2024], hybrid quadratic-linear transformers [Irie et al., 2025], retrieval-augmented generation [Lewis et al., 2020], and memory-augmented goal-directed agents [Wayne et al., 2018]. On the neuroscience side it examines cognitive maps in the hippocampal-entorhinal system [Whittington et al., 2020, Rambaud et al., 2025], the duality between hippocampal episodic memory and prefrontal working memory [Whittington et al., 2025], and the proposal that key-value memory unifies the two [Gershman et al., 2025].

The throughline of the review is that a small number of mathematical primitives recur across these literatures, and that the most informative recent results emerge from the connections between papers rather than from any single one. Two primitives do most of the work: an outer-product memory that binds keys to values and is read by a query, introduced in Chapter 2 alongside the sequence models that instantiate it, together with

the primal/dual choice between compressing those associations into a matrix and keeping the items to attend over; and the discounted future-occupancy operator $(I - \gamma T)^{-1}$, developed in Chapter 4, that recurs in linear-attention masks, in successor representations, and in active-inference value computation. Chapter 2 covers the machine-learning literature and Chapter 3 the neuroscience, where the cognitive-map lineage (TEM $\rightarrow$ TEM-t $\rightarrow$ Tale of Two $\rightarrow$ MapFormer) forms the spine that links the two fields. Chapter 4 then draws the primitives together, arguing that the brain’s associative memory and the sequence model’s compressed state are instances of one key-value object, and Chapter 5 draws out the open problems this duality exposes and proposes a sequence of connected projects that address them, including a group-correct Bayesian posterior for cognitive-map path integration [Rambaud et al., 2025] and an extension of hierarchical active inference with successor representations [Rangarajan and Rao, 2026].

Chapter 2

MEMORY ARCHITECTURES IN SEQUENCE MODELS

Memory has long been central to neural sequence models. Through the early deep learning era, recurrent neural networks (RNNs), and especially the long short-term memory (LSTM) architecture [Hochreiter and Schmidhuber, 1997], treated memory as the hidden state. The entire past is compressed into a fixed-size vector that the network updates one step at a time, with gating controlling the flow of information. A parallel line built memory as a separate, addressable store. Early examples include associative memories, such as Hopfield networks [Hopfield, 1982] and Kohonen’s correlation-matrix memory [Kohonen, 1972], and fast-weight programmers that write transient associations into a weight matrix [Schmidhuber, 1992]. Later, neural Turing machines [Graves et al., 2014] and the differentiable neural computer [Graves et al., 2016] coupled a controller network to an external memory it learns to read from and write to. Several of these share a single outer-product structure, developed in Section 2.1. Across these designs the memory is bounded, and the past is compressed or addressed rather than stored verbatim.

The transformer [Vaswani et al., 2017] marked a departure from this bounded, recurrent memory. Rather than compressing the past into a state, it keeps every token and re-reads them through attention at each step. This made training fully parallel and gave exact access to any earlier token, and on the strength of it the transformer has become the dominant sequence model and the foundation of nearly all modern large-scale systems, across language, vision, and multimodal domains. And as the field has progressed, the prevailing recipe for improving these foundation models has been to scale them up, with larger parameter counts, more training data, and longer context windows, and it is this rapid scaling that pushed the cost of keeping everything to the foreground. Softmax attention re-reads the entire past at every step: it maintains a key-value cache that grows linearly with the sequence and costs quadratically in time, so both compute and memory become prohibitive as contexts

lengthen. The cache is itself a particular choice about how to remember, namely to store every token verbatim and search all of them on each query. As contexts grew, this choice became the limiting factor, and the difficulty of scaling it drove the search for alternatives: rather than retaining every token, a model can compress the past into a bounded internal state that it updates as it reads, trading exact recall for bounded cost. Much of this renewed interest in memory is in this sense a return to the older bounded designs, but now reconciled with what made attention effective, namely content-addressable key-value access. This reframes long-range sequence modeling as a memory-design problem, with three recurring questions, what to write, how to update it, and how to read it back, and that is the lens through which the architectures in this chapter are best understood. The same three questions recur, in different vocabulary, in the biological memory systems of Chapter 3.

The memory view is only one response to the quadratic-cost problem, and several others have been pursued. One line keeps attention but makes it cheaper. Sparse-attention models restrict which positions may attend, through fixed factorized patterns [Child et al., 2019] or patterns that combine sliding windows and global dependencies [Beltagy et al., 2020]. Low-rank methods project the keys and values to a small fixed dimension [Wang et al., 2020], and kernel methods approximate the softmax with random features [Choromanski et al., 2021]. A second line of work shortens the sequence itself, pooling tokens into coarser units at intermediate layers so that attention runs over fewer positions, as in funnel- and hourglass-style hierarchical transformers [Dai et al., 2020, Nawrot et al., 2022]. These approaches cut the cost of attending over the explicit context, but they still operate over a representation of all positions and do not, on their own, provide a bounded, content-addressable memory that is written and read across the whole sequence. The memory view of this chapter instead replaces the growing context with a fixed-size state updated recurrently. This view and the sub-quadratic-attention line meet at linear attention: the kernel linearization that makes a Performer sub-quadratic is exactly what lets attention be rewritten as a fixed-state recurrence [Katharopoulos et al., 2020], so the most aggressive way to make attention cheap is already a way to compress the past into a bounded memory.

2.1 Outer-product memory: one primitive, many names

A recurring object across the architectures in this chapter is a memory matrix updated by outer products of (key, value) pairs and read by inner products with queries. Concretely, given a stream of (k_t, v_t) pairs, the memory is

$$W_t = W_{t-1} + v_t \otimes k_t,$$

and is read by

$$\hat{y}_t = W_t q_t = \sum_{\tau \leq t} (k_\tau^\top q_t) v_\tau.$$

This same object appears under many names: Kohonen’s correlation-matrix memory [Kohonen, 1972], the Fast Weight Programmer [Schmidhuber, 1992], the recurrent form of linear attention [Katharopoulos et al., 2020], the hidden state of a structured state-space model in its scalar- A specialization [Dao and Gu, 2024], and even the gradient-descent-trained linear layer when viewed through its dual form [Irie et al., 2022]. The equivalence between linear attention and fast weight programmers was made explicit by Schlag et al. [2021], and the broader fast-weight-programming line, together with its neurobiological connections, is reviewed by Irie and Gershman [2025]. Viewed yet another way, the outer product $v_t \otimes k_t$ is the binding operation of tensor-product representations [Smolensky, 1990], tying each value to its key as a role-filler pair and superposing the bindings in a single matrix; this is the connectionist lineage that links the object to the variable binding of the cognitive-map models in Chapter 3.

The last of these names, the gradient-descent-trained linear layer, is the least obvious, and Irie et al. [2022] make the identification precise. A linear layer trained by gradient descent accumulates its updates into the weight matrix, and each update is a rank-one outer product: at training step t , with input x_t and output gradient $(\nabla_y L)_t$, the update is $\Delta W_t = -\eta_t (\nabla_y L)_t x_t^\top = e_t \otimes x_t$, writing $e_t = -\eta_t (\nabla_y L)_t$ for the error signal. Over steps

$t = 1, \dots, T$ the weight is therefore $W = W_0 + \sum_t e_t \otimes x_t$. At test time,

$$S(x) = W_0 x + \sum_t (x_t^\top x) e_t,$$

so the layer’s output is an initial-weight term plus an unnormalized attention over the entire training history with training inputs as keys and error signals as values. The mathematics is not new (the identity goes back at least to Aizerman et al. [1964] and the representer theorem), but the framing emphasizes that nothing about “attention” or “key-value memory” is special to particular architectures: any linear projection over data trained by an outer-product-like rule is an instance. This generality is also the framing’s limitation. The equivalence is exact only for a linear layer, or per neuron with its input stream treated as given, since in a nonlinear network the inputs that serve as keys themselves drift during training; and the decomposition of a weight matrix into (key, value) pairs is not unique. The dual form is therefore best read as a unifying lens rather than a falsifiable mechanism, the same caution that applies to the key-value framework of Chapter 4.

Retrieval from this memory is clean only when the keys are mutually orthogonal. The read(out) recovers a stored value exactly when the query matches one key and that key is orthogonal to all the others; when keys overlap, it mixes in interfering values weighted by their key similarity. This is why dimensionality matters: in a high-dimensional space random keys are approximately orthogonal, so cross-talk is small, and pushing the keys to be still more separable, whether through a feature map, sparsity, or learning, buys cleaner retrieval at the cost of robustness. The same separability versus interference tension reappears as an explicit design choice in the key-value account of biological memory (Chapter 4), where keys are optimized for discriminability and similar memories are actively pushed apart to limit interference.

2.2 State-space duality: Mamba-2

Primal and dual forms. The read above can be computed in two equivalent ways (Figure 2.1). The *primal* form compresses the accumulated outer products into the single matrix W_t and reads in one matmul: recurrent, linear in time, and bounded in memory, the al-

algorithmic structure of linear attention and of an SSM. The *dual* form instead keeps every (k_τ, v_τ) pair and reads as an attention sum: quadratic and growing in memory, but with exact access to individual items. The two are exactly equal when the read is bilinear in (k, q) . Softmax attention [Vaswani et al., 2017] breaks this equality, because $\text{softmax}(QK^\top)V$ cannot be reassociated into a compressed matrix without an infinite-dimensional kernel feature map. Its most developed instance lives in structured state-space models.

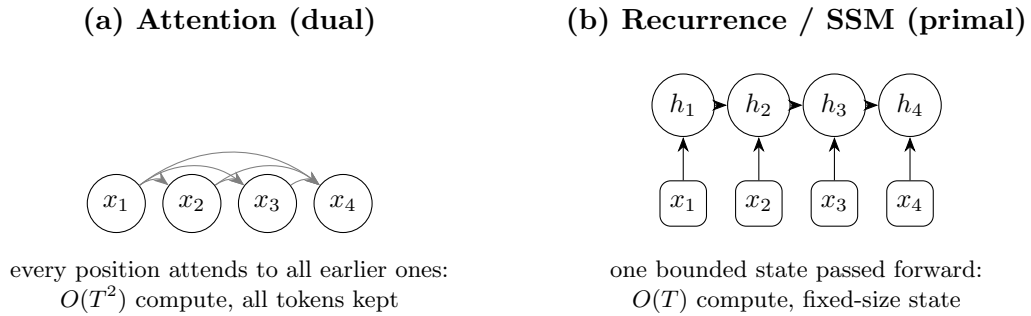


Figure 2.1: Two ways to handle the past. **(a)** The dual, keep-and-attend form connects every position to all earlier ones, so the work grows quadratically and every token must be kept. **(b)** The primal, compress form folds the past into a single bounded state h_t passed forward step by step, so the work is linear and the state stays a fixed size.

Structured state-space models. A state-space model maps an input sequence to an output through a linear latent recurrence: in continuous time $h'(t) = Ah(t) + Bx(t)$, $y(t) = Ch(t)$, and discretized with a step size Δ , $h_t = \bar{A}h_{t-1} + \bar{B}x_t$, $y_t = C^\top h_t$ with $\bar{A} = \exp(\Delta A)$. Structured state-space models such as S4 [Gu et al., 2022] brought this recurrence to long-range sequence modeling, using a fixed, HiPPO-based initialization of A that biases the dynamics toward long-range structure and set the early state of the art on the Long Range Arena benchmark. The transition matrix A carries the state, the model’s memory, from one step to the next, and most of the design space lies in how A is structured. A dense A is the most expressive but costs $O(N^2)$ per step and admits no parallel scan. Constraining it to be diagonal, as in Mamba-1 [Gu and Dao, 2024], lets each state dimension decay independently with its own rate, reducing the step to elementwise products and a linear-time scan. Mamba-2 constrains it further to a scalar, $A_t = a_t I$, sharing one gate across all state dimensions at

each step. This scalar case is the most restrictive, but it is exactly the structure under which the duality with masked linear attention holds; the resulting expressivity cost returns in the caveats below, and more sharply in the cognitive-map results of Chapter 3. In the selective SSMs of Mamba, B , C , and Δ are made input-dependent, so the recurrence chooses what to write and how fast to forget.

Dao and Gu [2024] establish a theoretical framework, State Space Duality (SSD), connecting structured SSMs to masked linear attention through semiseparable matrices. Unrolling the SSM recurrence $h_t = A_t h_{t-1} + B_t x_t$, $y_t = C_t^\top h_t$ gives a matrix transform $y = Mx$ where $M_{ji} = C_j^\top (A_j \cdots A_{i+1}) B_i$.

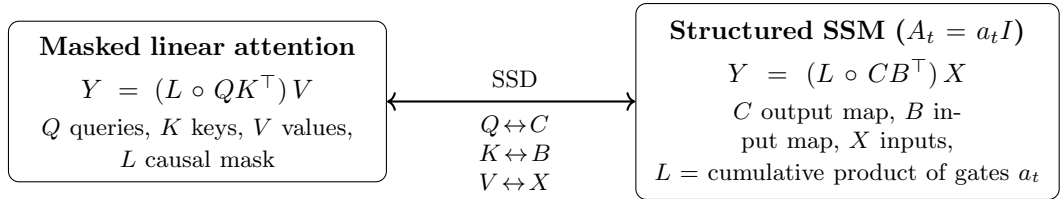


Figure 2.2: Under the scalar- A restriction, a structured state-space model and masked linear attention compute the same matrix transform. The mask L is the cumulative product of the scalar gates a_t , and the SSM’s output, input, and value matrices play the roles of the attention queries, keys, and values. Adapted from the State Space Duality of Dao and Gu [2024].

The matrix M is N -semiseparable: every off-diagonal block has rank at most N , with the $N = 1$ case (scalar gates $A_t = a_t I$) reducing to a cumulative product. Multiplication by a 1-SS matrix is exactly the scalar SSM scan, equivalently a generalization of cumsum where each term carries a decaying weight.

The duality holds in the special case $A_t = a_t I$, where the mask L reduces to the 1-semiseparable cumulative product of the scalar gates. Under this restriction the SSM’s matrix transform is identical to masked kernel attention, with the SSM’s parameters taking the roles of attention’s queries, keys, and values (Figure 2.2). Critically there is no softmax; the equivalence relies on the multilinear tensor contraction that softmax breaks.

The framework also gives a converse result (Theorem 5.2): any masked-attention model that supports efficient bounded-order autoregressive evaluation must have a semiseparable

mask, so the SSD-family is in a sense the class of efficient autoregressive masked attention.

The chunkwise SSD algorithm exploits the semiseparable structure to compute the matrix transform in $O(TN^2)$ matmul-dominated FLOPs (Figure 2.3). The sequence is block-decomposed into $Q \times Q$ chunks; diagonal blocks are computed via the quadratic (attention) form, off-diagonal blocks via the low-rank (semiseparable) form, with chunk-boundary states passed via a short scan over T/Q chunks. This is the same intra-chunk-parallel / inter-chunk-recurrent skeleton that re-appears in DeltaNet training [Yang et al., 2024b] and in the Blending paper’s hybrid memories.

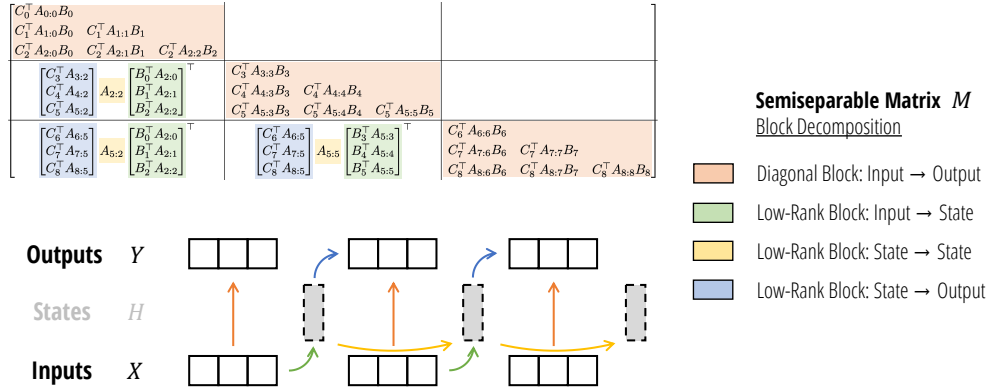


Figure 2.3: The chunkwise SSD algorithm. The causal transform $y = Mx$ is block-decomposed into T/Q chunks of length Q . Diagonal blocks are computed directly in the quadratic (attention) form; off-diagonal blocks are computed in the low-rank (semiseparable) form, with the chunk-boundary state carried forward by a short scan over the T/Q chunks. This intra-chunk-parallel, inter-chunk-recurrent schedule is the skeleton reused in DeltaNet training and the Blending hybrids. Reproduced from Dao and Gu [2024].

The duality also has clear limits. It holds between scalar- A SSMs and 1-SS masked linear attention, not between SSMs and softmax attention, and the scalar- A restriction is a real expressivity loss compared to Mamba-1’s per-channel diagonal A , since every channel then shares a single decay rate per step. Empirically, pure Mamba-2 is comparable to pure Transformer++, but a layer-wise hybrid (Mamba-2 with $\sim 10\%$ attention layers) beats both, which suggests the expressivity gap is real but narrow [Dao and Gu, 2024].

2.3 Blending complementary memory systems

Within a single layer, Irie et al. [2025] combine the two halves of the primal/dual duality: softmax attention (“KV-memory”), the keep-and-attend dual form, and a fast-weight / DeltaNet memory (“FW-memory”), the compressed primal form. They then systematically compare three ways of blending the two. The two memory types are complementary: KV-memory is precise, quadratic, and supports exact retrieval over a bounded recent window, while FW-memory compresses an unbounded history into a fixed-size state, lossy and linear-time but with expressivity, such as state tracking, that softmax attention provably lacks in a single forward pass.

The FW-memory used is DeltaNet, which replaces the pure Hebbian update with a delta rule:

$$W_t = W_{t-1} + \sigma(\beta_t) (v_t - W_{t-1}\phi(k_t)) \otimes \phi(k_t).$$

The factor $(v_t - W_{t-1}\phi(k_t))$ is a prediction error, so memory becomes *rewritable* (corrects stale associations) rather than append-only. With $\sigma = 2 \cdot \text{sigmoid}$, the state-transition matrix $(I - \sigma(\beta_t)\phi\phi^\top)$ can have eigenvalues in $(-1, 1)$ including negative values. Parity and modular arithmetic look elementary but are canonical state-tracking tasks, and they are not easy for the standard architectures: neither a fixed-depth softmax transformer nor a scalar- or diagonal- A SSM can compute them in a single forward pass [Merrill et al., 2024]. Grazzi et al. [2025] show that negative eigenvalues are exactly what is required for one-pass parity and modular arithmetic: scalar positive decay can integrate and forget but cannot sign-flip; negative eigenvalues unlock the 1-D sign-flip operation underlying parity. The two complementary update mechanisms are data-dependent gating, which decays the whole state for rapid erasure (gated linear attention, Yang et al., 2024a; and Mamba-2), and the delta rule, which corrects a single key’s value. Yang et al. [2025] fuse them (Gated DeltaNet), pairing rapid memory erasure with targeted updates and surpassing both Mamba-2 and DeltaNet, while Siems et al. [2025] take several delta steps per token through products of Householder transformations (DeltaProduct), giving diagonal-plus-rank- n transitions that solve harder S_3 and S_4 group word problems.

The three blends (Figure 2.4) differ in how KV- and FW-memory share information:

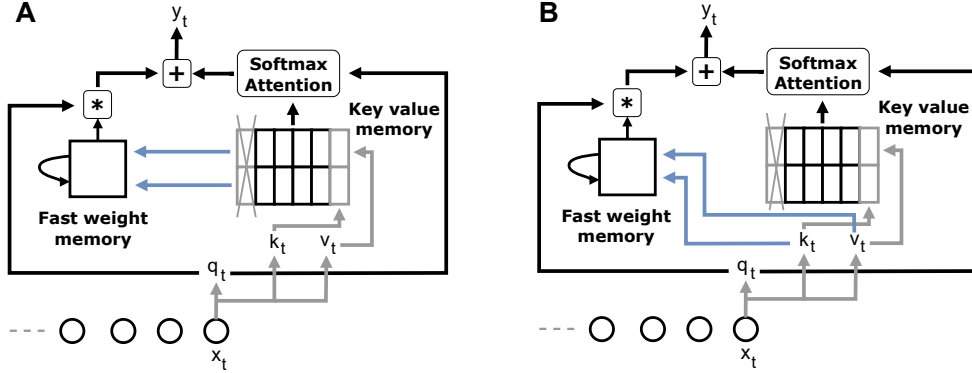


Figure 2.4: Three ways to blend KV-memory (softmax attention) with FW-memory (DeltaNet) within a layer: Delayed-Streaming, Delayed-Chunk, and Synchronous. Reproduced from Irie et al. [2025].

- *Delayed-Streaming*: KV holds the recent S tokens; when a token leaves the window it is written to FW. Division of labor by age (recent = KV, old = FW).
- *Delayed-Chunk*: FW is updated only at chunk boundaries; intra-chunk uses softmax. Equivalent in structure to Infini-attention [Munkhdalai et al., 2024].
- *Synchronous*: both memories receive (k_t, v_t) in real time. The architectural shape of Based [Arora et al., 2024] but with DeltaNet rather than vanilla linear attention.

Outputs are mixed via a dynamic per-dimension vector gate (the default) which adds $\sim 25\text{M}$ parameters at 340M scale.

The experiments probe each form of complementarity in turn. On general language modeling, all three blends are roughly equivalent to Transformer++ and DeltaNet, with Synchronous winning LAMBADA perplexity by $\sim 15\%$ relative. On expressivity tasks (parity, modular arithmetic), transformer fails and DeltaNet solves; the delayed variants *fail* because the delay destroys real-time state tracking, while Synchronous solves them. On retrieval (FDA / SWDE / SQuAD), a large KV window is required and FW-memory does not substitute for it. On a partially-observed RL visual-match task, the hybrid, using a quadratic-attention window of only 64, largely closes the gap to a full-context transformer.

Where and when the FW-memory is fed its input matters as much as which FW-memory is used. The Synchronous blend preserves the negative-eigenvalue state tracking that de-

layed variants destroy; the precision-vs-expressivity tradeoff is mediated by injection timing. These conclusions come with caveats: the expressivity tasks are synthetic, and on real language modeling all blends are roughly equal; there is no comparison to layer-wise hybrids such as Samba, Griffin, or Jamba; and the largest scale tested is 1.3B parameters.

2.4 *Memory as test-time learning*

A recent reframing makes the fast-weight idea explicit: the memory state is itself a small model, and writing to it is a step of learning performed at inference. Sun et al. [2025] make the hidden state a linear model or a two-layer MLP updated by a self-supervised loss at test time (TTT-Linear and TTT-MLP), and unlike a fixed-state recurrence these layers keep improving as context grows. Behrouz et al. [2025] (Titans) extend this with a learned long-term memory module that updates at test time via a gradient-based surprise signal, alongside short-term attention and persistent parameters. This view also supplies an alternative unifying lens to the outer-product algebra of Section 2.1: Wang et al. [2025] show that linear attention, state-space models, fast-weight programmers, online learners, and softmax attention are all *test-time regression* layers, performing memorize-then-retrieve and differing only in the regression weights, the regressor class, and the optimization algorithm. Concretely, each key plays the role of a regression input and its value the target, so writing to memory fits a predictor to the accumulated pairs and reading evaluates that predictor at the query. One reading locates the shared structure in the outer-product algebra; the test-time-regression framework locates it in the regression objective instead. The two are complementary descriptions of the same family.

2.5 *Retrieval-augmented generation: external symbolic memory*

Lewis et al. [2020] combine a parametric memory (knowledge in pre-trained LM weights) with a non-parametric memory (an external dense vector index of raw text); see Figure 2.5. The retrieved document z is treated as a latent variable and marginalized over.

Two variants differ in granularity: RAG-Sequence uses one document for the whole output; RAG-Token uses a different document per token, giving per-token soft attention over the retrieved set. Training is end-to-end on negative marginal log-likelihood without

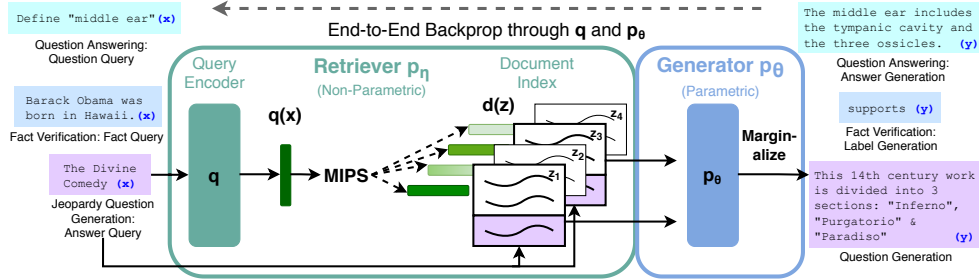


Figure 2.5: Retrieval-augmented generation: a query encoder retrieves passages from a frozen document index, and a seq2seq generator conditions on them as a marginalized latent. Reproduced from Lewis et al. [2020].

supervision on which document to retrieve; the document encoder and index are frozen and only the query encoder and the BART seq2seq backbone are fine-tuned.

Two related architectures position the same external store at different layers. kNN-LM [Khandelwal et al., 2020] freezes a trained LM, builds a datastore of (key = context representation, value = next token), and at test time interpolates the LM’s predicted distribution with a kNN distribution over the datastore, so memory enters at the *output*. RETRO [Borgeaud et al., 2022] chunks the input and integrates retrieved neighbors via chunked cross-attention at *mid-network* layers. Ordered by where the retrieved memory enters, these methods trace the same loose-to-tight integration axis that the Blending paper found decisive for fusing FW-memory with KV-memory (Figure 2.6): kNN-LM’s output interpolation is the loosest coupling, RAG’s input concatenation the tightest, and RETRO’s mid-network cross-attention lies between.

The RAG family occupies a distinctive and internally consistent corner of the memory design space. It is the canonical large-scale instance of the keep-items-and-attend, or dual, side of the memory primitive: the datastore is a set of (key, value) pairs read by similarity, with no compression into a matrix. Pushed to this scale, it makes a recurring claim vivid, that more memory helps only when retrieval is good enough and storage is cheap relative to encoding; kNN-LM is the deployed proof, since a smaller model with a larger datastore can beat a model that learned the same content into its parameters (Irie’s Remark 3.2). What RAG gives up to reach this scale is structure: where Vector-HaSH [Chandra et al., 2025] relies on a fixed structured scaffold and the cognitive-map and successor-representation

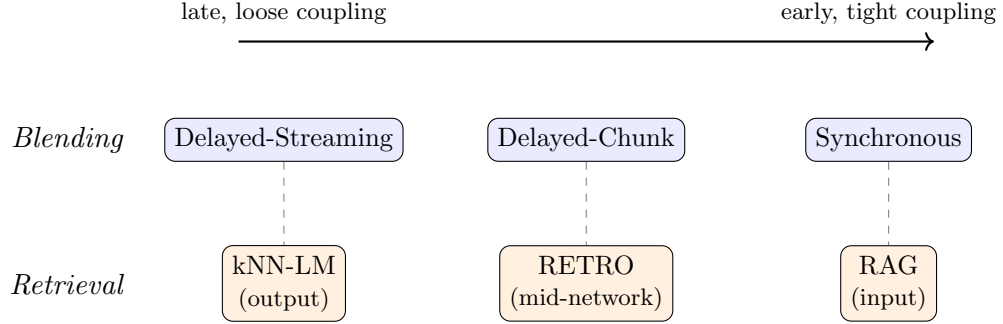


Figure 2.6: The retrieval-augmentation methods and the Blending schemes populate one axis: how early and tightly a secondary memory is integrated with the main computation. kNN-LM interpolates only the output distribution (loosest, latest), like the Delayed-Streaming blend; RETRO injects retrieved chunks through mid-network cross-attention, like the chunk-level Delayed-Chunk blend; RAG concatenates documents at the input so they are attended throughout (tightest, earliest), like the Synchronous blend.

models [Whittington et al., 2020, Rambaud et al., 2025] learn their structure, RAG learns none, trading it for scale and editability. The same trade reappears in its values, which are symbolic rather than distributed: RAG stores raw text, so the store is interpretable and can be hot-swapped to update world knowledge, but it is brittle to retrieval misses, where a top- k failure produces hallucination with none of softmax’s graceful degradation. RAG thus sits at the verbatim end of the arc this chapter opened on: where the early bounded memories compressed or addressed the past rather than storing it word for word, and the transformer keeps every token verbatim within its context, RAG moves verbatim storage outside the model altogether, into an editable external index.

2.6 Memory in goal-directed agents: MERLIN

Wayne et al. [2018] introduce a memory-augmented RL agent in which memory contents are formed by an unsupervised predictive objective rather than by reward (Figure 2.7).

The central thesis is that a large memory is not enough: the right information must be stored in the right format, and reward is the wrong teacher for that over long delays. Value functions propagate reward for value estimation across long horizons, but the gradient that shapes what to *write* still flows through the recurrence and is bounded by the backpropagation window τ ; once the delay between encoding and the dependent decision exceeds τ , the agent cannot learn what to store. MERLIN uses $\tau = 20$ (~ 1.3 s) to solve tasks up to

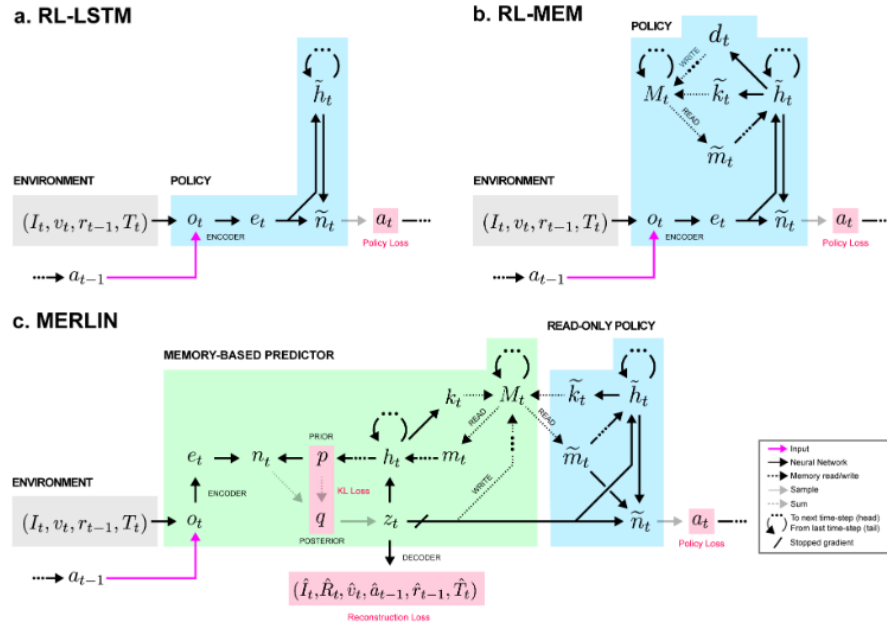


Figure 2.7: MERLIN: a memory-based predictor writes compressed state variables to an external memory; a read-only policy queries that memory to act. Reproduced from Wayne et al. [2018].

six minutes long.

The architectural pieces are:

- A *memory-based predictor* (MBP), a sequential VAE that serves as a *world model*, a learned model of how the environment evolves. A prior $p(z_t \mid \text{history-in-memory})$ predicts the next latent; a posterior q corrects it with the observation; z_t is sampled from q . Training minimizes a variational lower bound (reconstruction + KL). The objective is temporally local, so memory contents are shaped at encoding without long-range reward credit.
- A *return-prediction bottleneck* (a la Gluck & Myers) that forces z to carry reward-relevant features without becoming reward-only.
- *Retroactive memory updating*: each memory row stores z_t alongside a γ -discounted sum of future state variables, a successor representation [Dayan, 1993]. This makes retrieval predictive, since pulling up a waypoint also returns a forecast of what follows.
- A read-only policy over z and memory, trained conventionally.

These design choices pay off in the experiments. On latent learning across a delay greater than τ (T-maze), MERLIN solves the task while end-to-end agents are at chance. Read-head specialization emerges spontaneously: across five trained agents, the three read heads divide into locations-ahead, sub-goal doorways, and the goal itself. Decoding analyses show that allocentric goal position is recoverable from the MBP state but not from convnet / RL-LSTM / RL-MEM states.

The other architectures in this chapter are defined by their storage mechanism; MERLIN, by the *objective* that shapes what is stored. Its predictive, reward-free loss is what makes the right information available across long delays. The concern is learnability rather than expressivity, achieved by relocating the learning signal into a local prediction loss: its multimodal reconstruction targets play a teacher-forcing-like role that keeps each step’s representation learning local and well-posed. The architecture also touches the algebraic thread running through these models. Its retroactive store, which augments each memory

with a γ -discounted sum of future states, is a trajectory-sampled estimate of the future-facing member of the resolvent family $(I - \gamma T)^{-1}$ (Chapter 4), whose past-facing member is Mamba-2’s causal mask. MERLIN is therefore the point where the sequence-model mask meets the successor-representation and cognitive-map literature.

Chapter 3

BIOLOGICAL MEMORY SYSTEMS

How does the brain hold on to experience, generalize from it, and use it to guide behaviour? Understanding memory is one of the oldest questions in neuroscience. Much of this capacity has been traced to the hippocampal-entorhinal system. Place cells in the hippocampus, neurons that fire whenever the animal occupies a particular location, gave the first neural evidence of a map of physical space [O’Keefe and Nadel, 1978]. However, the map is now understood far more broadly in the tradition of Tolman’s original *cognitive map* [Tolman, 1948] as a reusable code for the relational structure of an environment that supports flexible, inferential behaviour [Behrens et al., 2018]. An important line of work ties this code with a successor representation, a predictive map of expected future states [Stachenfeld et al., 2017], which is also further addressed in Chapter 4. This hippocampal machinery underlies episodic memory, whereas a complementary system in prefrontal cortex holds information in sustained neural activity, a working memory that can be manipulated on the fly.

Memory can be approached at many levels, from the synaptic plasticity and engram cells that hold a trace, through the systems-level consolidation that stabilizes it [Kumaran et al., 2016], to the taxonomy of episodic, semantic, and working memory. This chapter works at the computational level, reviewing recent models of what these systems compute, and finds that they share the mathematics of the sequence models of the previous chapter. The cognitive-map lineage that carries the argument runs from the Tolman-Eichenbaum Machine through its transformer recasting, then to the synaptic-versus-activity memory duality of the Tale of Two Algorithms, and finally to MapFormer. Across this lineage, the brain’s structure-content memory turns out to be the outer-product, Hopfield, and key-value object of linear attention, and the path-integrated location code a learned positional encoding. The same group dynamics appear in either synaptic weights or neural activity,

and the transition that holds the map must be invertible, a rotation for a toroidal map, which gate-only diagonal- and scalar- A SSMs provably lack.

3.1 Cognitive maps as learned positional codes: TEM and TEM-t

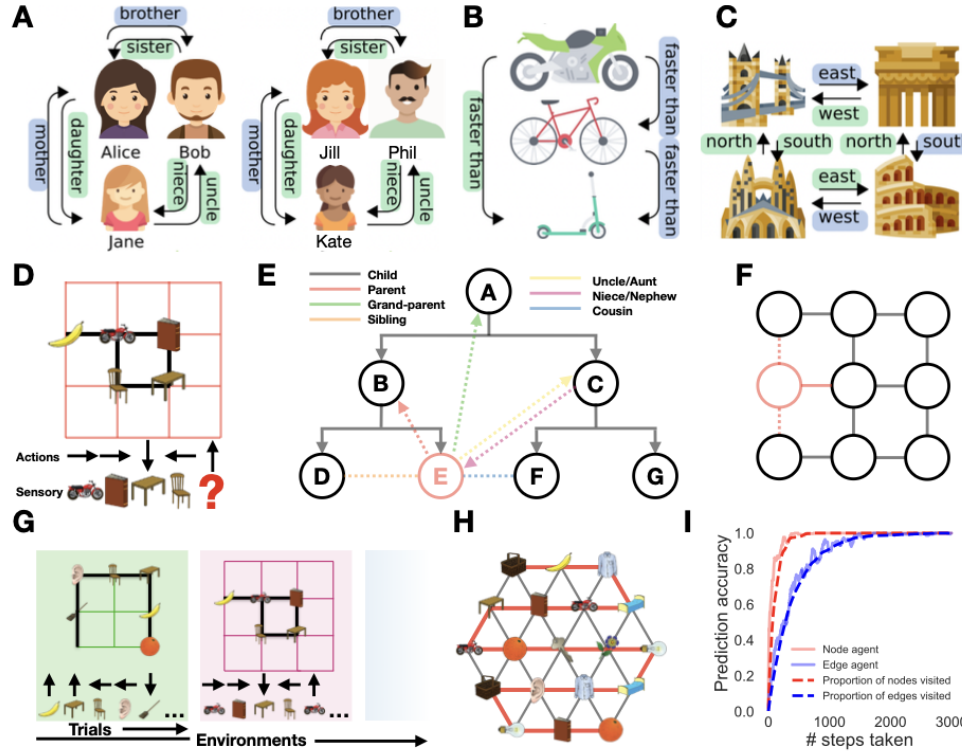


Figure 3.1: The Tolman-Eichenbaum Machine factorizes structure (an entorhinal path-integrated code) from content (a hippocampal binding of structure to sensory observations). Reproduced from Whittington et al. [2020].

One of the core papers framing this discussion is the Tolman-Eichenbaum Machine of Whittington et al. [2020], a model of the hippocampal-entorhinal system. It built on a single principle, memory should factorize *structure*, where one is, from *content*, what is there, so that structure learned in one environment transfers to another (Figure 3.1). A “where” module represents abstract position and updates it by path integration, applying an action-dependent transition to the current code, $g_{t+1} = \sigma(g_t W_a)$. Grid cells, entorhinal neurons whose firing fields tile the environment in a regular hexagonal lattice, and band

cells emerge in this module as a by-product of solving the task. That emergence is itself well understood: recurrent networks trained to path-integrate under a nonnegativity constraint develop hexagonal grid cells, which pattern-formation theory identifies as the optimal code for the task [Sorscher et al., 2023], and this is part of why the path-integration motif recurs throughout the lineage. A separate “what” stream carries the sensory content x at each location. The two are bound by an outer product, $p = \text{flatten}(x^\top g)$, a place-cell-like conjunction of position and stimulus, and these conjunctions accumulate in a fast-weight Hebbian memory $M = \sum_\tau p_\tau^\top p_\tau$ that is read by an attractor update $q \leftarrow \sigma(qM)$. Because every stored memory contains both the position g and the content x , retrieval can pattern-complete from either direction: a position cues its observation, and an observation cues its position. The model is trained self-supervised to predict the next observation across many environments that share an underlying structure but differ in how stimuli are arranged, so it is forced to learn that abstract structure rather than memorize any single environment. This is what lets it close loops zero-shot, inferring that right-down-left-up returns to the start, and transfer to non-spatial relational tasks such as transitive inference and family trees.

Whittington et al. [2022] then recast the model as a transformer and tie it more explicitly to the sequence-model literature. Keys and queries are computed from the position code alone and values from the stimuli alone, a causal mask is applied, and the position encodings are made recurrent and input-dependent, $e_{t+1} = \sigma(e_t W_a)$. A single attractor step, $q_t M_t = \sum_\tau (q_t \cdot p_\tau) p_\tau$, is exactly self-attention without the softmax, by the modern-Hopfield-equals-attention result of Ramsauer et al. [2021]. Writing each stored memory as the outer product $p = \text{outer}(\tilde{g}, \tilde{x})$, the read reduces to retrieving past content $\tilde{x}$ weighted by the similarity of position codes, $\tilde{x}_{\text{retrieved}} \approx \text{softmax}(\tilde{g}_t \tilde{G}^\top) \tilde{X}$, with the positions playing the role of keys and queries and the sensory content the role of values. The path-integrated code g is therefore the positional encoding itself, a location in a learned structure inferred on the fly. TEM-t is thus the explicit bridge from the neuroscience side, Hopfield networks, key-value memory, and fast weights, to attention.

3.2 The duality of synaptic and activity-based memory

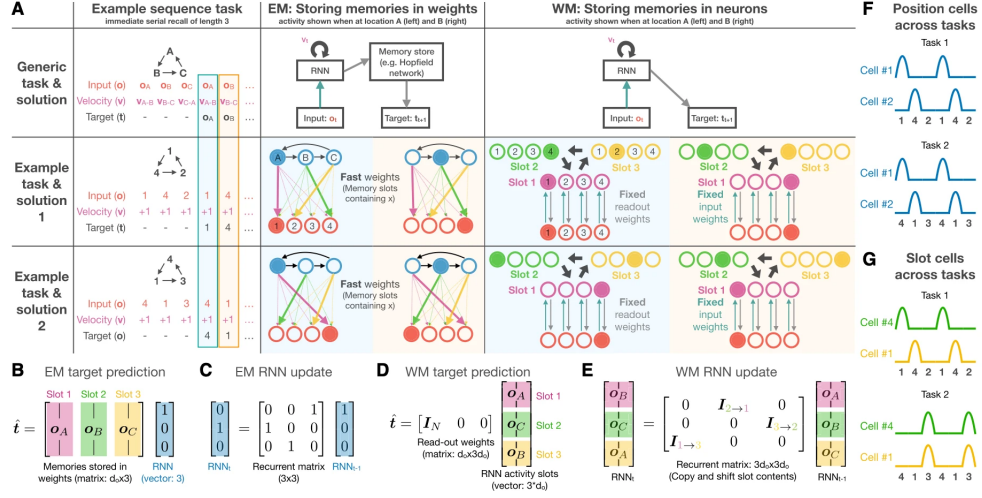


Figure 3.2: Two algorithms for the same computation: episodic memory holding associations in synaptic weights versus working memory holding them in controllable activity slots. Reproduced from Whittington et al. [2025].

The cognitive map need not be stored in synaptic weights alone. Whittington et al. [2025] establish a formal duality (Figure 3.2) between storing sequence memory in synaptic weights and storing it in neural activity, identifying hippocampal episodic memory (EM) and prefrontal working memory (WM) as two algorithms that compute the same function. Beyond the duality itself, the work offers a theory of prefrontal working memory as a set of controllable activation slots, one that unifies several otherwise disparate prefrontal recordings as a single slot mechanism reading from different sources of a velocity signal.

Both algorithms are derived in a structure-learning setting. Sequences are drawn from a fixed latent structure with positions z and observations hung at each position, and the velocities v that move between positions compose and cancel like a group, so that north, east, south, and west sum to zero and returning to a position returns its observation. The velocity operators form a group representation with inverses, $W_v W_{-v} = I$; on a closed loop they are literally discrete rotations, and under mild constraints the optimal position code is a grid-cell basis. This is exactly the rotation, or block-skew-symmetric, structure that the diagonal- A class of SSMs cannot represent, now realized biologically.

The same structure admits two solutions that differ only in where the memory is held. The first, episodic memory, moves a pointer over fixed content. An RNN tracks abstract position through $h(z) = W_v h(z - v)$ with W_v a permutation or rotation; a memory matrix $M = \sum_z o_z e_z^\top$ binds observations to positions by Hebbian outer products; and readout is $\hat{y}_t = M h_t = o_{z(t)}$. Mechanically this is flexible attention, the moving position code, indexing fixed weight-slots, the modern-Hopfield and linear-attention object of Section 2.1, with the memory matrix itself acting as a fast weight rewritten per task.

The second, working memory, rotates content past a fixed reader. The same equation is reshaped. The memory matrix is reinterpreted as a state vector $\text{vec}(M)$ whose columns become activation slots, each holding one observation. The readout weights are now fixed and read a single slot, with “slot 0” holding the current position’s observation, and the dynamics apply the expanded, transposed velocity operator, each 1 or 0 of W_v replaced by an identity or zero block to give a block-cyclic permutation P . The identity $\text{vec}(AXB) = (B^\top \otimes A)\text{vec}(X)$ accounts for the transpose: permuting the columns of M from the right appears as a transposed operator on the left of the state. Mechanically this is fixed attention, the readout, over flexibly rotating slots, with no synaptic plasticity once the task is learned.

The same group-rotation dynamics carry the same information in either substrate, fast weights in the episodic solution or fast activity in the working-memory one. In effect this is the primal/dual duality of the previous chapter recast as a substrate choice, the same associations held in a weight matrix or spread across activity slots.

The duality is not merely conceptual; it leaves a decoding fingerprint in neural activity, which makes it testable. Training linear decoders on RNN activity across many tasks separates the two solutions: episodic-memory solutions allow abstract position to be decoded and to generalize across tasks, whereas working-memory solutions do not. Working-memory solutions instead allow the slot sequence to be decoded, the observation that the copy-shift schedule places in each slot, but not the past sequence, the observation k steps ago; on constant +1 walks the two coincide, but varying the velocity pries them apart. With the appropriate velocity source, a working-memory architecture reproduces the prefrontal data the paper sets out to explain, including monkey serial recall, cued recall, and rodent sequential-goal tasks.

The duality comes with the usual caveats. The clean equivalence is between two simplified models, a one-hot RNN and a modern Hopfield network, and holds in the linear, simplified regime; the prefrontal-data explanations are post-hoc fits, and recovering the observed subspaces is not the same as uniquely identifying the mechanism; and the slot algebra is architecture-contingent, since compositionality holds for trajectory-independent recurrences rather than for activity-stored memory in general.

3.3 Parallelizing the cognitive map: MapFormer

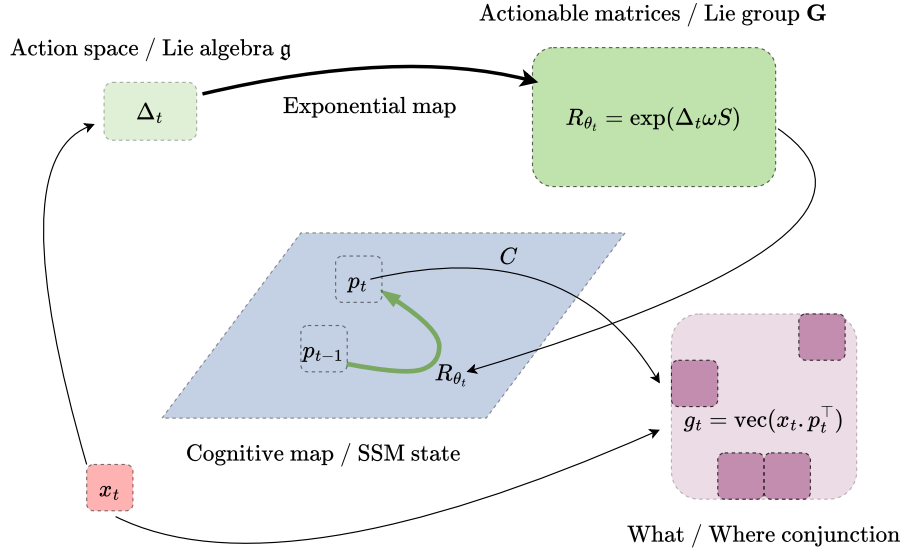


Figure 3.3: MapFormer path-integrates in the Lie algebra by a cumulative sum of input-dependent rotation angles, then exponentiates; the absolute-positional MapEM and rotary MapWM variants realize episodic and working memory. Reproduced from Rambaud et al. [2025].

Rambaud et al. [2025] complete the lineage, removing the sequential bottleneck that left the earlier models, TEM-t and the episodic and working-memory RNNs of the previous section, non-parallelizable, and supplying the group-theoretic foundation they lacked (Figure 3.3). The construction begins from a requirement on the transition itself. The action matrices that move the position code must form a *Lie group*, with closure, associa-

tivity, a null-action identity, and an inverse for every action. By the Peter-Weyl theorem any compact group decomposes into block-diagonal irreducible representations, which for a circle or a torus are $SO(2)$ rotations, so a cognitive map is held by *rotating* a position code rather than gating it. For efficiency, the path integration is carried out in the Lie algebra rather than the group. Each token is projected to an integration time Δ , a cumulative sum $\theta^{\text{PI}} = \omega \cdot \text{cumsum}_t(\Delta)$ is formed, and the result is exponentiated. Because $\exp(\sum_i A_i) = \prod_i \exp(A_i)$, this turns the group’s sequential matrix products into a parallel additive scan, and for 2×2 rotations no matrix exponential is even needed, only an explicit $\cos / \sin$ form applied RoPE-style. Making Δ input-dependent through a low-rank projection renders the path integration selective.

MapFormer comes in two variants, one for each side of the episodic and working memory duality of the previous section. MapEM, the episodic variant, uses absolute positional encoding, carrying a separate position population $p_t = R_{\theta_{0 \rightarrow t}} p_*$ and combining two parallel attentions, one for content and one for structure, by an element-wise product, as in TEM-t. MapWM, the working-memory variant, uses relative rotary positional encoding, a data-dependent form of RoPE [Su et al., 2024] that entangles structure and content directly in the rotated queries and keys. Three theoretical results follow. First, diagonal (Mamba [Gu and Dao, 2024]) and scalar (Mamba-2) SSM transitions cannot learn cognitive maps, because they can gate but not rotate, and a sigmoid gate has no inverse. Second, the SSM discretization $\bar{A} = \exp(\Delta A)$ is exactly the Lie exponential, so a structured SSM already path-integrates, only with the wrong, gate-only transition. Third, a linear RoPE transformer is precisely an SSM whose recurrent matrix is a 2×2 skew-symmetric rotation. Empirically, on selective-copy and forced-navigation tasks only the path-integrating models generalize out of distribution to longer or sparser sequences, and the episodic-memory variant scales more gracefully than the working-memory one, remaining more robust to neuron budget, sequence length, and vocabulary size.

The three results collapse into a single identity: the SSM discretization $\exp(\Delta A)$, the Lie exponential, the action matrix, and the cognitive-map step are the same operation, and a linear RoPE transformer is its 2×2 skew-symmetric instance. RoPE is then the fixed, non-selective special case of a working-memory cognitive map, and making its rotation angle

input-dependent turns a static positional code into a learned map on a graph or torus. This is genuine shared mathematics rather than analogy, and it joins the rotation thread of the sequence-model literature to the grid-cell rotation requirement, $W_v W_{-v} = I$, introduced in the previous section.

The lineage has real limits. TEM and TEM-t are bespoke, small-scale, and (for TEM-t) not parallelizable. MapFormer is causal-only, mostly limited to commutative rotation groups, and uses block-diagonal matrices without the biological change of basis, so it lacks the neural non-negativity that shapes grid codes [Sorscher et al., 2023]; it has not been scaled to language-model size, and its working-memory variant fails in higher dimensions. The scalar-SSM result is likewise narrow: it concerns structural out-of-distribution generalization, not language-model quality.

Chapter 4

BRIDGING BIOLOGICAL AND MACHINE MEMORY

Approached from opposite ends in the previous two chapters, biological and machine memory keep resolving to the same handful of mathematical objects. Building on that convergence, is the brain literally a key-value memory? This chapter weighs that claim against the literature that bears on it.

4.1 *Shared mathematical primitives*

Two mathematical objects do most of the work. The first, the outer-product memory, binds each value v_τ to a key k_τ by an outer product and accumulates the associations into one matrix,

$$M = \sum_{\tau} v_{\tau} k_{\tau}^{\top},$$

which a query q reads as $Mq = \sum_{\tau} v_{\tau} (k_{\tau}^{\top} q)$, a similarity-weighted sum of the stored values. It has appeared on both sides already, as the linear-attention and fast-weight state of Chapter 2 and as the hippocampal structure-content memory of Chapter 3, where a single attractor read turned out to be attention without the softmax. Because this one object underlies both the brain’s associative memory and the sequence model’s compressed state, the key-value account examined later in the chapter can speak to both fields at once. The second has surfaced only glancingly, in reinforcement learning, in sequence models, and in active inference, and rewards being set out on its own.

The resolvent $(I - \gamma T)^{-1}$. That second object is the resolvent of a transition operator. For a transition T and discount γ ,

$$(I - \gamma T)^{-1} = \sum_{k=0}^{\infty} \gamma^k T^k$$

is the discounted future-occupancy operator. Equivalently, it is the resolvent of T in the operator-theoretic sense, $(I - \gamma T)^{-1} = \frac{1}{\gamma}(\frac{1}{\gamma}I - T)^{-1}$, evaluated just outside the spectrum of T ; the series converges precisely when $\gamma \rho(T) < 1$, with $\rho(T)$ the spectral radius. For a diagonalizable $T = \sum_i \lambda_i u_i v_i^\top$ the operator is diagonal in the same eigenbasis,

$$(I - \gamma T)^{-1} = \sum_i \frac{1}{1 - \gamma \lambda_i} u_i v_i^\top,$$

acting on each eigenmode by the scalar $1/(1 - \gamma \lambda_i)$. This per-mode factor is where expressivity is won or lost. It is smooth accumulation for $\lambda_i \in (0, 1)$ but oscillates or sign-flips for negative or complex λ_i , exactly the gap between the gate-only transitions of diagonal- and scalar- A state-space models and the rotations a cognitive map requires (Chapter 3). Those rotational modes are biologically realized as the periodic grid code that path-integrating networks learn [Sorscher et al., 2023]. The transpose theme below is then immediate, since transposing conjugates these same modes: $((I - \gamma T)^{-1})^\top = (I - \gamma T^\top)^{-1}$. Many of the masks and memory operators seen so far are special cases. In Mamba-2’s State Space Duality, the causal mask L is the cumulative product of scalar gates, the resolvent’s lower-triangular special case with T a one-step shift [Dao and Gu, 2024]. In MERLIN, each memory row is augmented with a γ -discounted sum of *future* state variables, which is precisely the trajectory-sampled successor representation, the future-facing (upper-triangular) member of the same family [Wayne et al., 2018]. In hierarchical active inference, the successor representation $M = (I - \gamma T)^{-1}$ is used for value computation $v = Mr$ via the Bellman factorization that separates dynamics from reward [Rangarajan and Rao, 2026].

In each case the matrix is the same and only the read vector changes: reading the resolvent with a query gives memory retrieval, reading with a reward gives reinforcement-learning value, and reading with an expected free energy gives active-inference value. The vector you contract against is the objective. Active inference casts perception and action as minimizing variational free energy, a tractable bound on surprise, and scores plans by an expected free energy that adds an epistemic, information-gain term to reward.

This generalization motivates a recurring *transpose* theme. Lower-triangular vs upper-triangular masks correspond to causal vs anticausal evaluation; W_v vs $W_v^\top$ in cognitive-map

dynamics corresponds to moving a pointer over fixed content vs rotating content past a fixed reader (Chapter 3); and the future-facing successor representation sits opposite the past-facing linear-attention mask within the family. Flipping the direction of information flow reuses the same primitive.

A last cut runs orthogonal to these primitives and concerns where the memory physically lives. The same associative memory can be held in synaptic weights, as in the episodic solution of the Tale of Two Algorithms and in every fast-weight model, in sustained neural activity, as in its working-memory dual, or in an external non-parametric store, as in the retrieval-augmented family. The choice trades neuron count and plasticity against compute, parallelism, and editability, but the object stored is the same. A related cut concerns the form of the stored value. Retrieval-augmented systems keep raw, symbolic, human-editable text, whereas every other memory considered here stores distributed vectors, buying capacity and compositionality at the cost of interpretability. MapFormer refines the contrast by splitting the two, holding content as distributed vectors while actions live as operators in a low-dimensional group. These substrate and representation axes are where the open problems of Chapter 5 are posed.

Table 4.1 collects the two objects and the axes that organize them in one place. Each row is one structure the two fields arrived at independently, which is the sense in which the duality is structural rather than analogical.

4.2 *Key-value memory in the brain*

Gershman et al. [2025] argue that the brain implements key-value memory: keys (addresses, for retrieval) are represented separately from values (content, for storage), so each can be optimized for its own demand (Figure 4.1). Keys are pushed for discriminability; values are pushed for fidelity. The biological proposal is a division of labor in which hippocampus stores keys and neocortex stores values.

The technical framework unifies several memory models via a kernel-and-separation choice. The retrieval weights are $\alpha = \sigma(S(K, q))$, where S is a similarity kernel and σ is a separation operator. Different choices recover different models: softmax attention ($\sigma = \text{softmax}$), linear attention / FWP ($\sigma = \text{identity}$), Hopfield, sparse distributed memory ($\sigma =$

Table 4.1: The same primitives, as they appear on each side of the duality. Each row is one structure reached independently by both fields.

Primitive	In sequence models	In biological memory
Outer-product memory	Linear-attention / fast-weight state $M = \sum v k^\top$, read by a query	Hippocampal structure-content binding $M = \sum p^\top p$ (TEM), read by an attractor
Primal vs. dual read	Compressed recurrent state vs. attention over stored tokens	Episodic synaptic weights (EM) vs. working-memory activity slots (WM)
Transition structure	Scalar / diagonal A (Mamba-2) up to rotations (RoPE, negative-eigenvalue DeltaNet)	Grid-cell rotation $W_v W_{-v} = I$; SO(2) path integration
Resolvent $(I - \gamma T)^{-1}$	Mamba-2 causal mask; MERLIN’s retroactive store	Successor representation; active-inference value
Memory substrate	Weights vs. KV-cache vs. external store (RAG)	Synapses vs. sustained activity vs. neocortical store
Direction (transpose)	Causal vs. anticausal mask, L vs. $L^\top$	Pointer over content W_v vs. content past a reader $W_v^\top$
Value form	Distributed vectors (FWP) vs. symbolic text (RAG)	Distributed population codes; index vs. content

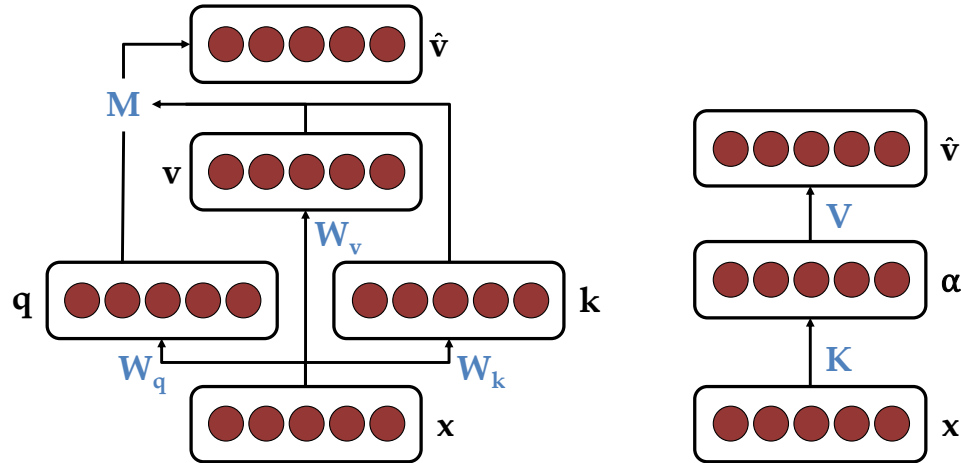


Figure 4.1: Two equivalent realizations of key-value memory: a correlation matrix built from Hebbian outer products, and an attention-style readout that scores a query against keys and combines the associated values. Reproduced from Gershman et al. [2025].

threshold), dense associative memory [Krotov and Hopfield, 2016] ($\sigma = \text{polynomial}$). The separation operator controls a separability-versus-robustness tradeoff: maximum separation is perfect but brittle; softmax sits in a tunable middle. Because the framework is this broad, “key-value memory” is used at two levels of generality: here it names the whole family, with fast-weight programming as the $\sigma = \text{identity}$ special case, whereas in the Blending hybrids of Chapter 2 it means softmax attention specifically. The same term thus names both the genus and one of its species.

The framework explicitly identifies linear layers as key-value memories via the dual form of Irie et al. [2022], and the correlation-matrix memory of Kohonen [1972] as the universal substrate. Biologically plausible learning rules for such memories have also been proposed: Tyulmankov et al. [2021] train key-value networks with a non-Hebbian, delta-rule-like update gated by global (neuromodulatory) and local (dendritic) signals, resembling hippocampal behavioral-timescale plasticity.

The paper makes three empirical claims: that memory is indelible and forgetting is a retrieval failure, that keys and values have distinct neural representations, and that keys are not consciously recallable. The supporting evidence spans the Shiffrin list experiments, which show interference rather than displacement; engram-cell experiments in which memory is recovered through optogenetic or cue-based reinstatement; the repulsion effect [Chanales et al., 2017], in which similar memories are pushed apart in hippocampal representation; and tip-of-the-tongue and feeling-of-knowing phenomena, in which a key match succeeds without value readout.

A computational simulation reproduces the silent-engram phenomenon. Training a key-value model on MNIST and then on FashionMNIST causes catastrophic forgetting on MNIST. The MNIST (key, value) terms remain in the weight matrix but the readout is overwhelmed by the FashionMNIST terms. Amplifying the MNIST terms by a scalar $\beta \geq 1$ restores MNIST performance without retraining, the computational analogue of optogenetic engram reactivation.

This account is the closest competitor to the synthesis developed here, which makes it worth marking where the two part ways. Three points stand out, and two of them open directly onto the proposals of Chapter 5.

The first is reconsolidation. The framework treats storage as essentially indelible, with forgetting recast as a retrieval failure, yet a large literature shows that retrieving a memory makes it labile and re-stores it in modified form, and that traces also degrade through synaptic decay and structural remodeling. A rewritable, error-driven memory captures this better than an append-only store. The DeltaNet update [Yang et al., 2024b] used in the hybrids of Irie et al. [2025] overwrites stale associations rather than accumulating them, and the slow transformation of an episodic trace into schematic knowledge, which the account does not model, is the consolidation problem taken up in Chapter 5.

The second is the relationship to cognitive maps. The separation TEM implements is structural versus sensory, medial versus lateral entorhinal cortex, rather than key versus value for a single item, so its hippocampus is autoassociative rather than the heteroassociative key-value store the framework describes. Vector-HaSH [Chandra et al., 2025] is the model that genuinely fits, while TEM and the Tale of Two Algorithms, the lineage the proposed work builds on, sit in a different position. Where the brain draws the key-value boundary, if it draws one at all, is therefore still open.

The third is a matter of falsifiability. Treating storage as indelible is hard to test, since storage and retrieval can be told apart only through a retrieval that succeeds or fails, and the kernel-and-separation formulation is flexible enough to absorb almost any memory model as a special case, with the repulsion effect its sharpest falsifiable prediction. The account also leans on a strict indexing role for the hippocampus, setting aside its reconstructive and constructive roles in imagination and future-thinking.

At the level of structure the duality is real, since the same outer-product memory and resolvent surface on both sides, which is what lets a key-value account address brains and sequence models at once. The stronger claim, that the brain is merely an indelible key-value store, needs more evidence. The next chapter will focus on the aspects that need to be addressed, such as reconsolidation, consolidation into schemas, and the cognitive-map boundary.

Chapter 5

FUTURE DIRECTIONS

The duality of the previous chapter also marks where the open problems lie. Read through the shared primitives and the axes that organize them, biological and machine memory leave the same four things unresolved: what an agent should write to memory, where that memory should live, how it consolidates into lasting structure, and whether any of it survives a long horizon. More research needs to be done to tackle these issues.

What gets written to memory is set by the objective that forms it. Reinforcement learning does propagate reward across long horizons, through value functions, bootstrapping, and dedicated distal-credit methods such as synthetic returns [Raposo et al., 2021] and memory-based value transport [Hung et al., 2019]. A scalar reward can prove to be a weak teacher for the long-delay *episodic* write, storing a specific cue now for a decision many steps later. There the gradient that shapes the write still passes through the recurrence and is truncated with the backpropagation window, and one distal reward barely disambiguates among the encodings that could have been stored. The limit is specific to this regime, since value-equivalent models such as MuZero learn useful control latents from reward and value alone [Schrittwieser et al., 2020]. Making the episodic write a prediction problem supplies the missing local signal. This is MERLIN’s move, and its value-equipped baselines underperformed the predictive objective on long-delay memory tasks [Wayne et al., 2018]. The recipe is now mainstream, from DreamerV3’s reconstruction-trained world model [Hafner et al., 2023] to R2I, which folds a structured state-space memory into that world model to improve long-term memory and long-horizon credit together [Samsami et al., 2024] and is the closest existing instance of the direction proposed below. Active inference shares the same locality on its perceptual side, where minimizing variational free energy is a per-step prediction error that shapes the generative model, but not on its action side, where scoring policies by expected free energy rolls the model forward over a horizon and inherits the

long-horizon attenuation that hierarchy is brought in to remedy. How to learn the right write rule when neither an external reward nor an obvious predictive target is available, and how to combine these objectives within a single agent, remains open. The DeltaNet write rule offers a related architectural angle, a rewritable, error-driven update that approximates a local prediction loss without an explicit world model.

The trade-off between precision, expressivity, and efficiency becomes relevant once both kinds of memory are present. Blending shows that real-time, synchronous injection of both memories preserves the expressivity of the fast-weight memory while delayed injection destroys it, and RAG, RETRO, and kNN-LM replay the same lesson at the parametric-versus-non-parametric scale. Whether these principles survive when the explicit context is itself a structured representation, a cognitive map or a scene graph rather than a flat token sequence, is open.

Consolidation lives on the structure-versus-scale axis, running from a fixed structured scaffold (Vector-HaSH), through learned cognitive maps (TEM, MapFormer, the Tale of Two Algorithms), to unstructured retrieval at scale (RAG). The successor representation is what makes reuse along it possible, since it holds the transition dynamics apart from the reward read against them, so a learned map can be reapplied to a new goal without being relearned. One open question is how schema consolidation interacts with hierarchical abstraction, namely when a recurring substructure should be lifted into a new “layer” of the cognitive map, and what a learning rule for that would look like. A further gap is that the reviewed models describe how memories are encoded, stored, and retrieved, but not how an episodic trace is slowly transformed into cortical, schematic knowledge. Predictive coding [Rao and Ballard, 1999], in which a hierarchical generative model predicts its own lower-level inputs and only the residual errors propagate upward, suggests a mechanism. If a cortical generative model continually predicts hippocampal replay, consolidation becomes the minimization of replay prediction error, so that repeatedly sampling a stored episode distills it into the schema that best explains it. This generative-replay reading of complementary learning systems [Kumaran et al., 2016] sits naturally on the active-inference machinery developed here, since the expected-free-energy reading of the resolvent $(I - \gamma T)^{-1}$ already treats memory-based value as prediction under a generative model, and it would

extend the account from the read-and-write operations the models handle well to the slower transformation between memory stores that they leave open.

Continual learning and long-horizon world modeling run into signal attenuation. Both the backpropagation-window argument of MERLIN and the discounted-horizon analysis of hierarchical active inference show that a signal must be locally usable to be learnable, and that coarse-graining through hierarchy is the shared remedy. Whether the hierarchical abstractions discovered by spectral clustering of the successor representation, as in hierarchical active inference, can be combined with the encoding objectives of MERLIN to produce world models that scale across both time and abstraction is the open question here.

These problems are connected, and so are the projects I propose to address them. Two threads of my own work meet in the middle, and together they outline a sequence in which each project supplies what the next one needs: a corrected cognitive map, a way to read that map with an objective, and a memory-complete agent built around the result.

Project 1: A group-correct Bayesian posterior for cognitive-map path integration. The cognitive-map lineage of Chapter 3 integrates actions open-loop: MapFormer’s location estimate $\theta^{\text{PI}} = \omega \cdot \text{cumsum}(\Delta)$ is pure dead reckoning, a prior with no correction step, so its per-step errors accumulate unchecked and the estimate drifts over a long sequence. The missing ingredient is a posterior. I have implemented a faithful MapFormer reproduction, both the absolute-positional (episodic) and rotary (working-memory) variants at paper-level accuracy, and grafted onto it a parallel, group-correct Invariant Extended Kalman Filter (InEKF): a content-only measurement head produces an innovation, a wrapped-innovation $\text{SO}(2)$ update applies the correction on the group, and the correction stays an affine recurrence $d_t = (1 - K_t) d_{t-1} + K_t \nu_t$ with a steady-state gain, so it remains a prefix-scan operation and preserves the $O(\log T)$ parallelism of the base model.

The same predict-then-correct structure recurs widely. The Kalman innovation ν is MERLIN’s posterior-minus-prior residual, the active-inference belief update, and the Rao-Ballard prediction error [Rao and Ballard, 1999]. The predict-then-update *ordering* is the invariant; the same ordering fix raised a faithful TEM reimplementation from 0.42 to 0.97 accuracy in my experiments. Adding the posterior is thus what structurally connects

the cognitive-map lineage (TEM $\rightarrow$ TEM-t $\rightarrow$ MapFormer) to the predictive-memory and active-inference lineage (MERLIN, hierarchical active inference, predictive coding). Because the correction is the same associative-scan primitive that makes state-space models parallel, the InEKF can be framed more broadly as a reusable, group-aware “filter layer” for transformers and state-space models, extending the parallel-scan story from state propagation to Bayesian correction.

On the lm200 cognitive-map task, a 64×64 torus with aliased observations and 200 unique landmarks, all models are trained at length $T = 128$ and evaluated out to $T = 2048$ in a fresh held-out environment (Figure 5.1). Adding the InEKF posterior to MapFormer’s open-loop path integration preserves its accuracy lead across the full out-of-distribution range, the empirical signature of the bounded-error Kalman correction; the open-loop MapFormer degrades faster as T grows, and a Mamba-like state-space model sits at the blank-prediction floor (~ 0.5) regardless of T , since diagonal state-space models cannot efficiently represent the $\text{SO}(2)$ rotations that path integration on a torus requires.

Project 2: Reading the corrected map with an objective. A clean cognitive map is only useful once it can be queried for behaviour. The natural next step is a successor-representation value readout $\nu = MG$ on top of the learned, posterior-corrected map: the same outer-product memory read by an objective vector, here an expected free energy, that the shared-primitives framework of Chapter 4 identifies. This turns the map from a passive estimate of where the agent is into a substrate for goal-conditional planning, and it is the explicit point where the path-integration thread joins the hierarchical active-inference thread of the next project. It also directly addresses the consolidation question, since a successor-representation readout exposes the structure-versus-content factorization that schema reuse depends on.

Project 3: Scaling hierarchical active inference into a memory-complete agent. My prior work on hierarchical active inference with successor representations [Rangarajan and Rao, 2026] is the planning instance of the resolvent $(I - \gamma T)^{-1}$ (Chapter 4) and already addresses long-horizon attenuation through hierarchy, with epistemic value as a built-in

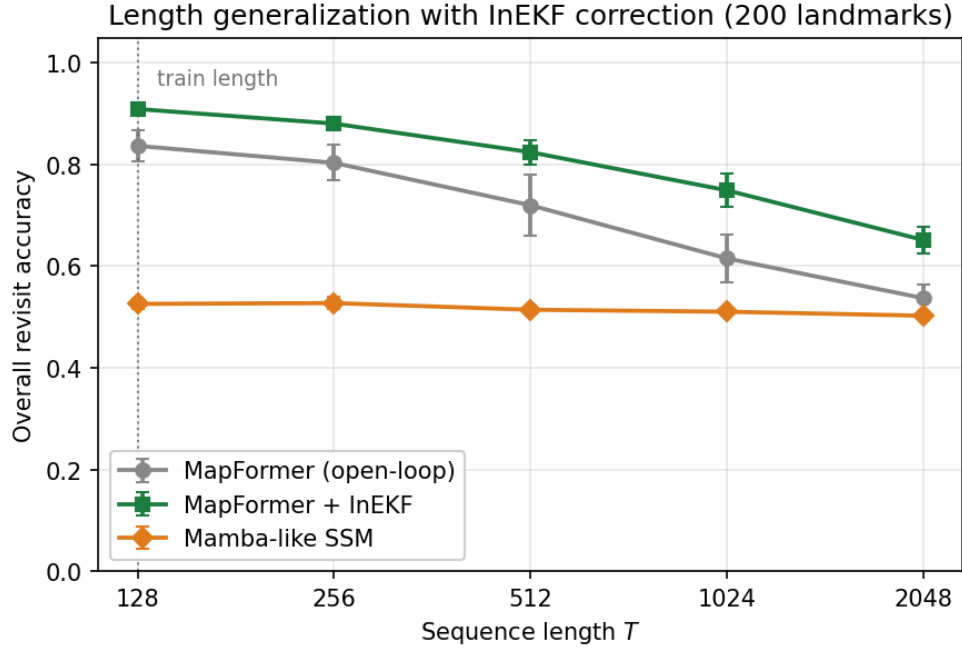


Figure 5.1: Length generalization on the lm200 cognitive-map task (64×64 torus, 16 aliased observation types, 200 unique landmarks). All models are trained at $T = 128$ (vertical dashed line) and evaluated on a fresh held-out environment up to $T = 2048$; bars are ± 1 SD across 3 seeds. The MapFormer + InEKF variant retains its train-time accuracy lead across the full out-of-distribution range, the empirical face of the bounded-error property of Kalman correction. The Mamba-like SSM, lacking input-dependent rotation in its path-integration mechanism, sits at the blank-prediction floor (~ 0.5) regardless of T : diagonal- A state-space models cannot efficiently represent the $SO(2)$ rotations needed for path integration on a torus [Rambaud et al., 2025, Appendix A.3].

ingredient the other reviewed models lack. The proposal is to extend it along the axes drawn out earlier. Its tabular successor representation could be scaled with the matmul-friendly state of Mamba-2 or the rewritable delta-rule update of DeltaNet (Chapter 2); a Tale-of-Two-style episodic or working memory (Chapter 3) could add the specific-trajectory recall that the forward-facing successor representation lacks; a RAG-style external store could supply editable goal and world specifications; and the encoding objectives of MERLIN and active inference could shape what the agent learns to write. Bounded-error closed-loop correction, from Project 1, and hierarchical coarse-graining here are complementary remedies for the two distinct long-horizon failure modes, drift and signal attenuation.

A successor-representation readout on a learned, posterior-corrected cognitive map is where the three projects meet: an agent whose internal memory compresses context efficiently, supports rapid binding and recall over long horizons, and exposes structured knowledge for revision, all coordinated by a single objective. The connecting principle is that internal memory is most useful when the substrate is matched to the time scale, the transition operator to the structure of the world, and the objective to what the agent must learn to write.

These projects also instantiate a broader commitment that runs through my work beyond cognitive maps. Its core is a predict-then-correct Bayesian filter whose innovation corrects a geometrically structured state, and it recurs across otherwise unrelated applications. In high-dimensional signal detection, coupled Kalman and particle filters with active, information-gain measurement selection track a low-rank subspace on a Grassmannian manifold, under the same $SO(2)$ structure as the cognitive-map InEKF above. In verifiable agents, the same innovation becomes a soft consistency check beneath retrieval-augmented and formal verification. Memory correction, geometric state estimation, and output verification are then one hierarchical, predict-then-correct inference read across settings.

Chapter 6

CONCLUSION

This report has surveyed recent literature on memory in sequence models and in the brain, organized around two complementary memory functions: context compression into a bounded working memory, and binding and episodic memory for fast generalization. The review identifies a small number of mathematical primitives that recur across architectures developed for very different purposes: outer-product fast-weight memory, read in either its primal (compressed) or dual (attend-over-items) form, and the resolvent operator that returns memory retrieval, reinforcement-learning value, or active-inference value according to the vector it is read with. It traces a hierarchy of state-transition structures that orders their expressivity, running from scalar decay through negative eigenvalues to rotation, and a distinction in memory substrate (fast weights versus fast activity) that the brain instantiates through the hippocampal-prefrontal division. It draws a key-value-memory framework across both fields, with a critical perspective on what that framework can and cannot claim.

The same connections make the remaining open questions more precise. The trade-off between explicit context and internal memory is mediated by where the memory is injected (Blending’s Delayed-to-Synchronous axis, replayed in the RAG family). Consolidation into long-term structural knowledge depends on the structure-versus-scale axis (cognitive maps versus unstructured retrieval at scale). Continual learning and long-horizon world modeling are bottlenecked by signal attenuation, for which hierarchy is the shared remedy. The encoding-objective axis, which runs from end-to-end reward through unsupervised prediction to unified free energy, supplies a way to ask what the agent should learn to write, not just how to write it.

The proposed dissertation develops two threads that meet in the middle. The first adds a group-correct, parallel-scannable Bayesian posterior to cognitive-map path integration, connecting the TEM-MapFormer lineage to the predict-then-correct structure shared by

MERLIN, hierarchical active inference, and predictive coding, with preliminary experiments showing that the corrected model holds its accuracy across out-of-distribution sequence lengths where open-loop path integration drifts. The second extends hierarchical active inference along the axes drawn out earlier: deep successor representations combining the value factorization of HAI with the matmul-friendly state of Mamba-2; episodic and working memory in the Tale of Two Algorithms sense; editable external knowledge in the RAG sense for goal specification; and encoding objectives drawn from MERLIN and active inference. A successor-representation readout on a learned, posterior-corrected cognitive map is where the two threads join. Across both fields the same lesson recurs. Memory earns its keep when its substrate fits the time scale, its transition operator fits the structure of the world, and its objective fits what the agent must learn to write.

BIBLIOGRAPHY

- Mark A. Aizerman, E. M. Braverman, and L. I. Rozonoer. Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control*, 25:821–837, 1964.
- Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.18668.
- Timothy E. J. Behrens, Timothy H. Muller, James C. R. Whittington, Shirley Mark, Alon B. Baram, Kimberly L. Stachenfeld, and Zeb Kurth-Nelson. What is a cognitive map? organizing knowledge for flexible behavior. *Neuron*, 100(2):490–509, 2018.
- Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. In *arXiv preprint*, 2025. arXiv:2501.00663.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint*, 2020. arXiv:2004.05150.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2022.
- Avi J. H. Chanals, Ashima Oza, Serra E. Favila, and Brice A. Kuhl. Overlap among spatial memories triggers repulsion of hippocampal representations. *Current Biology*, 27:2307–2317.e5, 2017.
- Sarthak Chandra, Sugandha Sharma, Rishidev Chaudhuri, and Ila Fiete. Episodic and associative memory from spatial scaffolds in the hippocampus. *Nature*, 638(8051):739–751, 2025.

- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint*, 2019. arXiv:1904.10509.
- Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2009.14794.
- Zihang Dai, Guokun Lai, Yiming Yang, and Quoc V. Le. Funnel-transformer: Filtering out sequential redundancy for efficient language processing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2006.03236.
- Tri Dao and Albert Gu. Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality. *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL <https://arxiv.org/abs/2405.21060>.
- Peter Dayan. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4):613–624, 1993.
- Samuel J. Gershman, Ila Fiete, and Kazuki Irie. Key-value memory in the brain. *Neuron*, 2025. URL <https://arxiv.org/abs/2501.02950>.
- Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.
- Riccardo Grazi, Julien Siems, Jörg K. H. Franke, Arber Zela, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear RNNs through negative eigenvalues. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2411.12537.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2312.00752.

- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2111.00396.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023. URL <https://arxiv.org/abs/2301.04104>.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- John J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.
- Chia-Chun Hung, Timothy Lillicrap, Josh Abramson, Yan Wu, Mehdi Mirza, Federico Carnevale, Arun Ahuja, and Greg Wayne. Optimizing agent behavior over long time scales by transporting value. *Nature Communications*, 2019. URL <https://arxiv.org/abs/1810.06721>.
- Kazuki Irie and Samuel J. Gershman. Fast weight programming and linear transformers: from machine learning to neurobiology. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2508.08435.
- Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. The dual form of neural networks revisited: Connecting test time predictions to training patterns via spotlights of attention. In *International Conference on Machine Learning (ICML)*, 2022. URL <https://arxiv.org/abs/2202.05798>.
- Kazuki Irie, Morris Yau, and Samuel J. Gershman. Blending complementary memory systems in hybrid quadratic-linear transformers. *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2506.00744>.

- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning (ICML)*, 2020.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.
- Teuvo Kohonen. Correlation matrix memories. *IEEE Transactions on Computers*, C-21(4): 353–359, 1972.
- Dmitry Krotov and John J. Hopfield. Dense associative memory for pattern recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1606.01164.
- Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- William Merrill, Jackson Petty, and Ashish Sabharwal. The illusion of state in state-space models. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2404.08819.
- Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.
- Piotr Nawrot, Szymon Tworkowski, Michał Tyrolski, Lukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. Hierarchical transformers are more efficient language

- models. In *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022. arXiv:2110.13711.
- John O’Keefe and Lynn Nadel. *The Hippocampus as a Cognitive Map*. Oxford University Press, 1978.
- Victor Rambaud, Sarah Mascarenhas, and Yair Lakretz. MapFormer: Self-supervised learning of cognitive maps with input-dependent positional embeddings. *arXiv preprint arXiv:2511.19279*, 2025. URL <https://arxiv.org/abs/2511.19279>.
- Hubert Ramsauer, Bernhard Schöfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Thomas Adler, Lukas Gruber, Markus Holzleitner, Milena Pavlović, Geir Kjetil Sandve, Victor Greiff, David Kreil, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. Hopfield networks is all you need. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/2008.02217>.
- Prashant Rangarajan and Rajesh P. N. Rao. Hierarchical active inference using successor representations. *Neural Computation*, 2026. URL <https://arxiv.org/abs/2604.15679>.
- Rajesh P. N. Rao and Dana H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1): 79–87, 1999.
- David Raposo, Sam Ritter, Adam Santoro, Greg Wayne, Theophane Weber, Matthew Botvinick, Hado van Hasselt, and Francis Song. Synthetic returns for long-term credit assignment. *arXiv preprint arXiv:2102.12425*, 2021. URL <https://arxiv.org/abs/2102.12425>.
- Mohammad Reza Samsami, Artem Zhohus, Janarthanan Rajendran, and Sarath Chandar. Mastering memory tasks with world models. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2403.04253>.
- Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning (ICML)*, 2021. arXiv:2102.11174.

- Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 2020. URL <https://arxiv.org/abs/1911.08265>.
- Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. DeltaProduct: Improving state-tracking in linear RNNs via householder products. In *arXiv preprint*, 2025. arXiv:2502.10297.
- Paul Smolensky. Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, 46(1-2):159–216, 1990.
- Ben Sorscher, Gabriel C. Mel, Samuel A. Ocko, Lisa M. Giocomo, and Surya Ganguli. A unified theory for the computational and mechanistic origins of grid cells. *Neuron*, 111(1):121–137.e13, 2023.
- Kimberly L. Stachenfeld, Matthew M. Botvinick, and Samuel J. Gershman. The hippocampus as a predictive map. *Nature Neuroscience*, 20(11):1643–1653, 2017.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024. arXiv:2104.09864.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2407.04620.
- Edward C. Tolman. Cognitive maps in rats and men. *Psychological Review*, 55(4):189–208, 1948.

- Danil Tyulmankov, Ching Fang, Annapurna Vadaparty, and Guangyu Robert Yang. Biological learning in key-value memory networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2110.13976.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. Test-time regression: a unifying framework for designing sequence models with associative memory. In *arXiv preprint*, 2025. arXiv:2501.12352.
- Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint*, 2020. arXiv:2006.04768.
- Greg Wayne, Chia-Chun Hung, David Amos, Mehdi Mirza, Arun Ahuja, Agnieszka Grabska-Barwińska, Jack Rae, Piotr Mirowski, Joel Z. Leibo, Adam Santoro, Mevlana Gemici, Malcolm Reynolds, Tim Harley, Josh Abramson, Shakir Mohamed, Danilo Rezende, David Saxton, Adam Cain, Chloe Hillier, David Silver, Koray Kavukcuoglu, Matt Botvinick, Demis Hassabis, and Timothy Lillicrap. Unsupervised predictive memory in a goal-directed agent. *arXiv preprint arXiv:1803.10760*, 2018. URL <https://arxiv.org/abs/1803.10760>.
- James C. R. Whittington, Timothy H. Muller, Shirley Mark, Guifen Chen, Caswell Barry, Neil Burgess, and Timothy E. J. Behrens. The Tolman-Eichenbaum machine: Unifying space and relational memory through generalisation in the hippocampal formation. *Cell*, 183(5):1249–1263.e23, 2020.
- James C. R. Whittington, Joseph Warren, and Timothy E. J. Behrens. Relating transformers to models and neural representations of the hippocampal formation. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2112.04035>.

- James C. R. Whittington, William Dorrell, Timothy E. J. Behrens, Surya Ganguli, and Mohamady El-Gaby. A tale of two algorithms: Structured slots explain prefrontal sequence memory and are unified with hippocampal cognitive maps. *Neuron*, 113(2):321–333.e6, 2025.
- Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning (ICML)*, 2024a. arXiv:2312.06635.
- Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b. URL <https://arxiv.org/abs/2406.06484>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.06464.